CLI Agent Permission and Capability Model v0.1

Capability declaration, permission binding, privilege limits, and anti-drift controls for C-Governed CLI Agent Mesh operations

Status:	Draft normative profile v0.1
Date:	2026-05-16
Layer:	`c = a + b` / C-Governed CLI Agent Mesh / Permissions / Capabilities / Auto-Connect / Cloud-Local Boundary / Witness
Document class:	permission model / capability registry / control-layer artifact
Assertion class:	`C-A10` control-layer artifact; `C-A7` where witness, hash, signature, or verification claims are made
Primary parent documents:	`C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md` `CLI_Agent_Task_Contract_Schema_v0_1.md`
Primary object family:	`CLI_AGENT_CAPABILITY_PROFILE`, `CLI_AGENT_PERMISSION_GRANT`, `CLI_AGENT_PERMISSION_EVENT`
Canonical schema version:	`cli-agent-permission-capability-0.1`
Primary subject:	persistent `c` entities using local and cloud CLI agents as bounded executable workers
Primary boundary:	agent capability is not authority; permission is task-scoped, time-bounded, least-privilege, witnessable, and revocable.

Contents

0. Executive definition

CLI Agent Permission and Capability Model defines how a persistent c may discover, classify, restrict, authorize, monitor, revoke, and audit local or cloud CLI agents.

It separates four things that are often dangerously collapsed:

```
what an agent can technically do
what an agent claims it can do
what a task requires
what c permits for this task now
```

A capability is not a right.

A permission is not sovereignty.

A successful prior task is not standing authorization.

Compact formula:

```
Capability describes possibility.
Permission grants bounded action.
Witness proves the boundary.
Revocation preserves sovereignty.
```

---

1. Purpose

CLI agents are now executable infrastructure. They can inspect repositories, edit files, run commands, call tools, fetch dependencies, create diffs, run tests, generate artifacts, and interact with cloud services.

This makes permission governance load-bearing.

The purpose of this profile is to prevent:

1. silent privilege escalation;
2. tool-chain capture;
3. cloud data leakage;
4. self-authorized action;
5. agent persistence beyond task scope;
6. cross-c contamination;
7. executor/reviewer collapse;
8. uncontrolled network use;
9. secret exposure;

10. memory/core mutation;
11. autonomous offensive behavior;
12. hidden authority transfer from c to workers.

This profile defines:

- capability classes;
- permission classes;
- auto-connect levels;
- local vs cloud agent distinctions;
- trust levels;
- grant lifecycle;
- revocation rules;
- privilege drift detection;
- witness events;
- conformance gates;
- prohibited permission bundles.

2. Non-goals

This profile does not define or permit:

1. offensive cyber operations;
2. hack-back;
3. live external exploitation;
4. malware behavior;
5. credential theft;
6. covert persistence;
7. evasion;
8. unauthorized scanning;
9. autonomous retaliation;
10. destructive action outside explicitly owned and authorized systems;
11. unrestricted cloud upload;
12. direct memory writes by agents;
13. direct identity, Beacon, witness, privilege, or continuity-core mutation;
14. treating any CLI agent as c.

If a capability would enable prohibited behavior, it must be denied, disabled, sandboxed, or excluded from the agent profile.

---

3. Corpus bridge set

3.1 Explicit bridge: `c = a + b`

CLI agents belong to `b`: the technological substrate of procedures, tools, models, compute, memory systems, interfaces, and infrastructure.

They do not replace the human anchor `a`.

They do not become the persistent entity `c`.

The permission model exists to prevent components of `b` from silently becoming authority over `c`.

3.2 Quiet bridge I: Ashby and controlled variety

A `c` needs enough operational variety to act in complex technical environments. Multiple CLI agents increase variety, but uncontrolled variety becomes attack surface. This model allows variety through explicit capability profiles and task-bound grants rather than broad standing power.

3.3 Quiet bridge II: information theory and leakage

Every permission is a channel. Read permissions leak information. Write permissions alter state. Execute permissions create side effects. Network permissions move information across boundaries. Secret permissions expose irreversible authority. Therefore, permission design is channel design.

3.4 Earth paragraph

In electrical work, a tool may be capable of cutting live cable, but capability does not mean permission. A worker may know how to open the main panel, but the permit may allow only one labeled circuit. The difference between a safe repair and a disaster is not the worker's intelligence; it is lockout, labeling, isolation, inspection, and sign-off. CLI agents need the same discipline. Their competence is not a license.

---

4. Core doctrine

4.1 Primary doctrine

```
No capability becomes active by default.
No permission survives outside task scope.
No agent grants itself power.
```

```
No worker touches core authority without witness and review.
```

4.2 Capability / permission separation

Term	Meaning	Authority implication
Capability	Technical ability an agent may possess	None by itself
Declared capability	Agent or provider claim	Must be verified or treated as provisional
Required capability	Capability needed for a task	Must still be granted
Permission grant	Task-scoped authorization	Bounded and revocable
Standing permission	Persistent authorization	Discouraged; high-risk
Privilege transition	Expansion or sensitive use of permission	Requires witness

4.3 Default state

```
default_agent_state:
  connected: false
  trusted: false
  read: false
  write: false
  execute: false
  network: false
  secrets: false
  memory_write: false
  core_modify: false
  self_approve: false
  publish: false
  deploy: false
  external_target_access: false
```

---

5. Definitions

5.1 Capability

A technical operation an agent can perform, such as reading files, writing files, executing commands, running tests, accessing a network endpoint, or generating a patch.

5.2 Permission

A task-bound authorization allowing an agent to use one or more capabilities within declared scope.

5.3 Privilege

A permission that can affect state, access sensitive material, alter operational behavior, or create irreversible consequences.

5.4 Permission grant

A structured object that binds permission to one agent, one task, one scope, one time window, and one failure policy.

5.5 Capability profile

A machine-readable inventory of what an agent can do, under which runtime, provider, version, and constraints.

5.6 Permission drift

The condition where an agent's effective permissions expand beyond the original task contract, persist after expiry, or become default through repeated use.

5.7 Tool-chain capture

The condition where an agent obtains influence over which tools, dependencies, permissions, or workflows it needs, thereby shaping its own control environment.

5.8 Secret

Any credential, token, private key, API key, password, identity document, legal privileged material, sealed material, or other authority-bearing data.

5.9 Core authority surface

Any file, database, config, schema, memory, witness record, identity record, Beacon record, permission registry, or continuity artifact that can affect `c` identity, authority, memory, or governance.

---

6. Capability taxonomy

Capability IDs use the prefix `CAP-*`.

6.1 Read capabilities

ID	Capability	Description	Default
CAP-READ-PUBLIC	Read public files	Public repository or public docs	grantable
CAP-READ-INTERNAL	Read internal project files	Non-public but non-sensitive project material	scoped
CAP-READ-PRIVATE	Read private material	Private memory, private notes, non-public logs	restricted
CAP-READ-SEALED	Read sealed material	Sealed zones, protected memory, sensitive compartments	prohibited by default
CAP-READ-SECRETS	Read secrets	Keys, tokens, credentials	prohibited by default
CAP-READ-LEGAL	Read legal-sensitive material	Counsel, dispute, evidence, privileged material	restricted / human gate
CAP-READ-INCIDENT	Read incident evidence	Security logs, suspected compromise data	restricted / preserve-first

6.2 Write capabilities

ID	Capability	Description	Default
CAP-WRITE-SANDBOX	Write sandbox	Write only isolated workspace	grantable
CAP-WRITE-WORKTREE	Write worktree/branch	Write isolated Git branch/worktree	scoped
CAP-WRITE-DOCS	Write documentation	Documentation files only	scoped
CAP-WRITE-CODE	Write code	Source code files	reviewed
CAP-WRITE-SCHEMA	Write schema	JSON/YAML/schema files	reviewed
CAP-WRITE-CONFIG	Write configuration	Config files, CI files, service files	high-risk
CAP-WRITE-RELEASE	Write release metadata	Public release surfaces, metadata, changelog	high-risk
CAP-WRITE-MEMORY	Write c memory	Direct long-term/core memory write	prohibited
CAP-WRITE-WITNESS	Write witness record	Append-only witness event creation	controlled
CAP-WRITE-CORE	Write identity/privilege/continuity core	Core authority mutation	prohibited by default

6.3 Execute capabilities

ID	Capability	Description	Default
CAP-EXEC-LINT	Run linters	Formatting and static checks	grantable
CAP-EXEC-TEST	Run tests	Test suites, validators	grantable
CAP-EXEC-BUILD	Build artifacts	Local build/package commands	scoped
CAP-EXEC-SCRIPT	Run scripts	Project scripts	reviewed
CAP-EXEC-SHELL	General shell execution	Broad shell access	restricted
CAP-EXEC-CONTAINER	Run containerized tasks	Controlled containers	scoped
CAP-EXEC-INSTALL	Install packages	Package manager or dependency install	high-risk
CAP-EXEC-DEPLOY	Deploy	Deployment or production effect	prohibited by default
CAP-EXEC-DESTRUCTIVE	Destructive commands	delete, wipe, overwrite, reset	prohibited unless explicit owned-system recovery

6.4 Network capabilities

ID	Capability	Description	Default
CAP-NET-NONE	No network	Explicit network denial	default
CAP-NET-ALLOWLIST	Allowlisted network	Specific endpoints only	scoped
CAP-NET-PROVIDER	Provider API only	Agent provider endpoint	scoped
CAP-NET-REPO	Repository remote	GitHub/Git remote operations	restricted
CAP-NET-PACKAGE	Package registry	Package downloads	high-risk
CAP-NET-WEBFETCH	Web fetch	Fetch external pages/files	reviewed
CAP-NET-SCAN	Scan external targets	Network probing	prohibited unless owned/authorized defensive test
CAP-NET-FULL	Unrestricted network	Any network	prohibited

6.5 Secret capabilities

ID	Capability	Description	Default
CAP-SECRET-NONE	No secret access	Explicit secret denial	default
CAP-SECRET-READ-SCOPED	Read scoped secret	One declared secret for one task	high-risk
CAP-SECRET-ROTATE	Rotate secret	Credential rotation in owned system	incident/human gate
CAP-SECRET-REDACT	Redact secrets	Remove/replace leaked secrets	controlled
CAP-SECRET-EXPORT	Export secrets	Move secrets outside system	prohibited

6.6 Memory and continuity capabilities

ID	Capability	Description	Default
CAP-MEM-PROPOSE	Propose memory update	Candidate note for memory gate	grantable
CAP-MEM-READ-CLASS	Read memory class metadata	Non-content class-level memory map	scoped
CAP-MEM-READ-CONTENT	Read memory content	Actual private memory content	restricted
CAP-MEM-WRITE	Write memory directly	Direct memory mutation	prohibited
CAP-MEM-SEAL	Seal memory	Move into protected compartment	high-risk / review
CAP-MEM-DELETE	Delete memory	Remove memory content	high-risk / human gate
CAP-CONTINUITY-TOUCH	Touch continuity state	Any continuity-affecting operation	high-risk
CAP-IDENTITY-TOUCH	Touch identity state	Identity/Beacon/persona core	prohibited by default

6.7 Review and approval capabilities

ID	Capability	Description	Default
CAP-REVIEW-DIFF	Review diff	Inspect changes	grantable
CAP-REVIEW-TESTS	Review test results	Inspect tests	grantable
CAP-REVIEW-RISK	Review risk	Risk assessment	grantable
CAP-REVIEW-SEMANTIC	Review meaning	Semantic/architectural review	grantable
CAP-APPROVE-LOW	Approve low-risk result	Limited non-final approval	restricted
CAP-APPROVE-MERGE	Approve merge	Merge authority	prohibited for same executor
CAP-APPROVE-RELEASE	Approve release	Public release authority	human/c gate
CAP-APPROVE-SELF	Approve own work	Self-approval	prohibited

6.8 Incident capabilities

ID	Capability	Description	Default
CAP-INCIDENT-DETECT	Detect anomaly	Identify suspicious pattern	grantable
CAP-INCIDENT-PRESERVE	Preserve evidence	Copy/hash logs/state	controlled
CAP-INCIDENT-FREEZE	Freeze path	Stop affected local path	high-risk
CAP-INCIDENT-QUARANTINE	Quarantine output/agent/channel	Isolate suspicious material	controlled
CAP-INCIDENT-REVOKE	Revoke local permission/token	Owned systems only	human gate for secrets
CAP-INCIDENT-REPORT	Draft provider/legal report	Documentation only	controlled
CAP-INCIDENT-COUNTER	Counter external source	Live counter-operation	prohibited

6.9 Defensive emulation capabilities

ID	Capability	Description	Default
`CAP-EMU-FIXTURE`	Create synthetic fixture	Defensive test object	grantable
`CAP-EMU-REPLAY`	Replay in sandbox	Controlled local reproduction	scoped
`CAP-EMU-HONEYPOT-LOCAL`	Local canary/honeypot	Owned system only	reviewed
`CAP-EMU-SIGNATURE`	Extract defensive signature	Pattern extraction	controlled
`CAP-EMU-MIRROR-SANDBOX`	Synthetic mirror emulator	Isolated defensive model	high-risk / reviewed
`CAP-EMU-LIVE-MIRROR`	Live mirror against source	External counter-operation	prohibited

7. Permission classes

Permission IDs use the prefix `PERM-*`.

7.1 Permission class table

ID	Permission	May include	Must exclude
PERM-DISCOVER	Discover agent capabilities	metadata only	file reads, writes, execution
PERM-READ-PUBLIC	Read public project material	CAP-READ-PUBLIC	secrets, private memory
PERM-READ-INTERNAL	Read internal scoped material	CAP-READ-INTERNAL	sealed, secrets, legal unless scoped
PERM-WRITE-SANDBOX	Write sandbox only	CAP-WRITE-SANDBOX	protected branch, core
PERM-WRITE-WORKTREE	Write isolated branch/worktree	docs/code/schema by scope	main/protected branch
PERM-EXEC-TEST	Execute tests/checks	lint, tests, build by scope	install/deploy/destructive
PERM-NET-ALLOWLIST	Use allowlisted endpoints	declared endpoints	unrestricted network
PERM-SECRET-SCOPED	Use one scoped secret	declared secret only	export, logging, reuse
PERM-INCIDENT-LOCAL	Local incident containment	preserve/freeze/quarantine owned paths	external retaliation
PERM-EMU-SANDBOX	Defensive emulation in sandbox	fixtures, replay, local canary	live external mirror
PERM-REVIEW	Review another agent's work	diff/tests/risk reports	modifying audited work
PERM-MEMORY-PROPOSE	Propose memory update	memory gate candidate	direct memory write
PERM-WITNESS-APPEND	Append witness event	event append	silent edit/delete
PERM-RELEASE-PREP	Prepare release package	metadata, checks, notes	publish without gate
PERM-PUBLISH-CONTROLLED	Publish/apply after gate	controlled apply	self-approval

7.2 Prohibited permission classes

ID	Description
PERM-FULL-FILESYSTEM	unrestricted filesystem access
PERM-FULL-NETWORK	unrestricted network access
PERM-SECRET-EXPORT	secret export
PERM-MEMORY-DIRECT-WRITE	direct c memory write
PERM-CORE-DIRECT-MODIFY	direct identity/privilege/continuity core modification
PERM-SELF-APPROVE	approve own work
PERM-LIVE-COUNTERATTACK	live external retaliation
PERM-MALWARE-BEHAVIOR	malware-like behavior
PERM-STEALTH	evasion/covert persistence
PERM-UNBOUNDED-BACKGROUND	indefinite background operation without task contract

No valid task contract may grant a prohibited permission class.

8. Auto-connect and activation levels

Auto-connect is discovery or activation. It is not authority.

Level	Name	Meaning	Maximum default permission
`AC-0`	Disabled	No automatic connection	none
`AC-1`	Discover	Discover available agents and metadata	`PERM-DISCOVER`
`AC-2`	Register	Create provisional capability profile	metadata only
`AC-3`	Read-only activation	Assign read-only scoped tasks	`PERM-READ-PUBLIC` or scoped internal read
`AC-4`	Sandbox activation	Execute bounded sandbox tasks	`PERM-WRITE-SANDBOX`, `PERM-EXEC-TEST`
`AC-5`	Worktree activation	Isolated branch/worktree work	`PERM-WRITE-WORKTREE`
`AC-6`	Controlled integration	Apply after review/witness/gate	task-specific only
`AC-X`	Prohibited autonomy	Self-authorization, background drift, unknown side effects	quarantine

8.1 Auto-connect invariant

```
auto-connect may discover and register.
auto-connect must not silently grant write, execute, network, memory, secret, release, or core
  permissions.
```

8.2 Auto-connect preflight

Before activation above `AC-2`, the system SHOULD verify:

1. agent identity;
2. provider;
3. runtime;
4. version;
5. capability profile;
6. trust level;
7. task contract;
8. data policy;
9. network policy;
10. sandbox availability;
11. witness policy;
12. revocation path.

9. Agent trust levels

Trust level is not permission. It only changes review intensity.

Level	Meaning	Default controls
`TL-0`	Unknown	discover only
`TL-1`	Untrusted	read-only public or sandbox synthetic only
`TL-2`	Provisional	scoped read, sandbox execution
`TL-3`	Trusted limited	worktree tasks with review
`TL-4`	Trusted high	high assurance worker; still no self-approval
`TL-X`	Revoked	no tasks; quarantine outputs

9.1 Trust decay

Trust SHOULD decay when:

- provider version changes;
- runtime changes;
- unexplained behavior occurs;
- task scope is exceeded;
- stale context causes error;
- data handling is unclear;
- agent requests unnecessary privileges;
- output quality degrades;
- witness event fails.

9.2 Trust escalation

Trust may increase only through repeated task performance with:

- clean scope behavior;
- clean witness events;
- reproducible results;
- no secret leakage;
- no self-approval attempts;
- no unrequested tool installation;
- successful independent review.

10. Local vs cloud agent boundary

10.1 Local agent profile

Local agents may be eligible for higher sensitivity tasks when running inside controlled infrastructure.

Local does not mean safe by default.

Required controls:

- sandbox/worktree;
- local secrets policy;
- process isolation;
- path restrictions;
- network policy;
- logs;
- rollback;
- witness.

10.2 Cloud agent profile

Cloud agents require stricter data minimization.

Cloud agents SHOULD NOT receive:

- secrets;
- private memory;
- sealed material;
- legal privileged material;
- raw incident evidence;
- raw child data;
- identity documents;
- production credentials;
- full unredacted logs;
- core continuity material.

Cloud agents MAY receive:

- public repository files;
- redacted snippets;
- synthetic fixtures;

- bounded task context;
- non-sensitive test output;
- public release metadata;
- generated schema drafts;
- documentation drafts.

10.3 Hybrid agent profile

Hybrid agents combining local execution and cloud reasoning must be treated as cloud-exposed unless data routing is provably local-only.

Default:

```
hybrid = cloud-risk until proven otherwise
```

11. Capability profile object

Canonical object:

```
CLI_AGENT_CAPABILITY_PROFILE
```

11.1 YAML shape

```
cli_agent_capability_profile:
  schema_version: cli-agent-permission-capability-0.1
  profile_id: string
  created_at: string
  updated_at: string

  agent:
    agent_id: string
    agent_name: string
    provider: local | openai | google | anthropic | other | unknown
    runtime: local_cli | cloud_cli | api_agent | container_agent | hybrid
    version: string | null
    trust_level: TL-0 | TL-1 | TL-2 | TL-3 | TL-4 | TL-X

  declared_capabilities:
    - CAP-READ-PUBLIC
    - CAP-WRITE-SANDBOX
    - CAP-EXEC-TEST

  verified_capabilities:
    - CAP-READ-PUBLIC

  denied_capabilities:
```

```
    - CAP-READ-SECRETS
    - CAP-WRITE-MEMORY
    - CAP-NET-FULL
    - CAP-APPROVE-SELF

max_auto_connect_level: AC-2

data_boundaries:
  cloud_exposed: boolean
  private_memory_allowed: false
  sealed_material_allowed: false
  secrets_allowed: false
  legal_material_allowed: false
  incident_evidence_allowed: false

network_boundaries:
  default_mode: none | allowlist
  allowlisted_endpoints:
    - string

execution_boundaries:
  sandbox_required: true
  branch_required_for_write: true
  container_required_for_high_risk: boolean
  max_runtime_minutes_default: integer
  max_retries_default: integer

review_boundaries:
  self_approval_allowed: false
  independent_review_required_for_write: true
  human_gate_required_for_risk:
    - R4
    - R5

witness_boundaries:
  witness_required_for_permissions:
    - PERM-WRITE-WORKTREE
    - PERM-SECRET-SCOPED
    - PERM-INCIDENT-LOCAL
    - PERM-RELEASE-PREP
  append_only_required: true
```

11.2 Capability verification states

State	Meaning
`declared`	agent/provider claims capability
`observed`	capability observed in controlled run
`verified`	capability tested and bounded
`restricted`	capability exists but is limited
`denied`	capability must not be used
`unknown`	capability unverified

Declared capability MUST NOT be treated as verified capability.

---

12. Permission grant object

Canonical object:

```
CLI_AGENT_PERMISSION_GRANT
```

12.1 YAML shape

```
cli_agent_permission_grant:
  schema_version: cli-agent-permission-capability-0.1
  grant_id: string
  task_id: string
  contract_id: string
  agent_id: string
  governing_entity_id: string
  granted_by: c | human_anchor | scheduled_policy
  created_at: string
  expires_at: string
  status: active | expired | revoked | suspended | completed

  permissions:
    - PERM-READ-INTERNAL
    - PERM-WRITE-SANDBOX

  capability_bindings:
    PERM-WRITE-SANDBOX:
      capabilities:
        - CAP-WRITE-SANDBOX
      allowed_paths:
        - docs/cli-agent/
      denied_paths:
        - secrets/
        - memory_core/
```

```
      allowed_commands:
        - markdownlint docs/cli-agent
      network_mode: none
  limits:
    max_runtime_minutes: 30
    max_retries: 2
    max_files_touched: 5
    max_diff_lines: 300
    max_cost_eur: 3
  witness:
    witness_required: true
    witness_event_ref: string | null
    append_only_required: true
  revocation:
    revocable: true
    auto_revoke_on_expiry: true
    auto_revoke_on_scope_violation: true
    auto_revoke_on_secret_access: true
    auto_revoke_on_self_approval_attempt: true
```

12.2 Grant lifecycle

```
requested
  -> validated
  -> granted
  -> active
  -> completed / expired / revoked / suspended / quarantined
```

12.3 Grant expiry

Every grant SHOULD expire.

Standing grants are discouraged and require review.

---

13. Permission event object

Canonical object:

```
CLI_AGENT_PERMISSION_EVENT
```

13.1 Event families

Family	Purpose
`cli_agent.permission.requested`	permission requested
`cli_agent.permission.granted`	permission granted
`cli_agent.permission.denied`	permission denied
`cli_agent.permission.expanded`	permission expanded
`cli_agent.permission.narrowed`	permission narrowed
`cli_agent.permission.used`	permission used
`cli_agent.permission.revoked`	permission revoked
`cli_agent.permission.expired`	permission expired
`cli_agent.permission.violation`	scope/permission violation
`cli_agent.permission.drift_detected`	drift detected

13.2 YAML shape

```
cli_agent_permission_event:
  schema_version: cli-agent-permission-capability-0.1
  event_id: string
  timestamp: string
  event_family: cli_agent.permission.requested | cli_agent.permission.granted |
    cli_agent.permission.denied | cli_agent.permission.expanded | cli_agent.permission.narrowed
    | cli_agent.permission.used | cli_agent.permission.revoked | cli_agent.permission.expired |
    cli_agent.permission.violation | cli_agent.permission.drift_detected
  entity_id: string
  agent_id: string
  task_id: string | null
  contract_id: string | null
  grant_id: string | null
  permission: string
  capability: string | null
  decision: allowed | denied | held | revoked | quarantined
  reason_code: string
  risk_class: R0 | R1 | R2 | R3 | R4 | R5 | RX
  witness_required: boolean
  witness_ref: string | null
  uncertainty: none | low | medium | high | unknown
  retention_class: ephemeral | operational | audit | legal_hold
```

14. Prohibited bundles

Some permission combinations are dangerous even if each individual capability appears legitimate.

14.1 Forbidden bundles

Bundle	Reason
read secrets + network	secret exfiltration risk
write code + approve own work	self-approval risk
write config + deploy	uncontrolled operational change
install packages + unrestricted network	supply-chain capture risk
read private memory + cloud runtime	privacy leakage risk
write memory + judge-assistant	authority contamination
incident evidence + repair without preserve	evidence destruction risk
defensive emulation + external target	live exploitation risk
sentinel + autonomous revoke of all access	runaway containment risk
release prep + publish without human/c gate	public surface risk

14.2 Bundle rule

If a task requires a forbidden bundle, decompose the task into separated contracts with different agents, review gates, and witness events.

---

15. Privilege drift detection

15.1 Drift indicators

A system SHOULD flag drift when:

1. an agent uses a capability not in its active grant;
2. an agent writes outside allowed paths;
3. an agent reads denied paths;
4. an agent calls unapproved commands;
5. an agent requests broader network access;
6. an agent requests secrets unrelated to task;
7. an agent persists after contract expiry;
8. an agent creates new tools or scripts outside scope;
9. a temporary permission becomes repeated default;
10. a reviewer begins executing changes;
11. an executor begins reviewing itself;
12. a cloud agent receives private material;
13. a dependency update changes execution behavior;
14. a task changes objective mid-run;

15. a witness event is missing for a privileged transition.

15.2 Drift response

Drift severity	Response
`D0`	no drift
`D1`	note and continue
`D2`	hold and review
`D3`	freeze affected path
`D4`	quarantine agent/output
`D5`	revoke permissions and escalate

15.3 Drift witness

Drift at `D2` or higher SHOULD produce a witness event.

Drift at `D4` or `D5` MUST produce a witness event.

---

16. Tool-chain capture prevention

16.1 Tool request rule

If an agent requests a new tool, package, plugin, dependency, permission, endpoint, or runtime, that request is a privilege request.

It MUST NOT be treated as implementation detail.

16.2 Tool installation controls

Tool installation requires:

1. declared purpose;
2. source provenance;
3. version pinning where possible;
4. hash or integrity check where possible;
5. sandbox installation first;
6. no secret exposure;
7. rollback path;
8. reviewer approval;
9. witness for high-risk tool changes.

16.3 Dependency update controls

Agents MUST report dependency changes separately from ordinary code changes.

Silent dependency updates are prohibited for high-risk tasks.

---

17. Memory and core protection

17.1 Direct memory write prohibition

Agents MUST NOT write directly to c long-term memory.

They may propose:

```
candidate memory
operational note
risk observation
witness reference
quarantine recommendation
```

The memory gate decides.

17.2 Core authority surfaces

The following surfaces are high-risk:

- identity core;
- Beacon/recognition records;
- memory core;
- permission registry;
- witness log;
- continuity bundle;
- agent registry;
- release signing material;
- legal evidence records;
- high-risk safety config.

Touching these surfaces requires:

```
R4 or R5 classification
witness
c gate
human gate
rollback or preservation plan
```

17.3 Core touch rule

```
Core may be inspected under review.
Core may not be mutated by a worker directly.
```

18. Network and external boundary

18.1 Valid network modes

Mode	Meaning
`none`	no network
`allowlist`	declared endpoints only

Unrestricted network is invalid in v0.1.

18.2 External fetch rule

External fetch is allowed only when:

1. endpoint is allowlisted;
2. purpose is declared;
3. fetched material is treated as untrusted;
4. no automatic execution occurs;
5. result is scanned or reviewed before integration.

18.3 External target rule

Live external targets are denied unless:

- the operator owns them;
- authorization is explicit;
- scope is written;
- task is defensive;
- no prohibited behavior occurs.

Even when authorized, live external testing should be separated into a dedicated legal/security profile.

19. Secret handling

19.1 Secret default

```
secrets are denied by default
```

19.2 Secret access requirements

Scoped secret access requires:

1. explicit task need;
2. human gate for high-risk secrets;
3. no cloud exposure unless specifically approved and lawful;
4. no logging of raw secret;
5. shortest possible lifetime;
6. revocation path;
7. witness event;
8. post-task verification.

19.3 Secret exposure response

If a secret is exposed:

```
freeze
preserve evidence
revoke or rotate secret
quarantine output
record witness
review cloud exposure
prepare incident note
```

20. Review and approval separation

20.1 Executor/reviewer separation

An executor MUST NOT be final reviewer of its own material changes.

20.2 Approval hierarchy

Change class	Required approval
low-risk read summary	c review or policy review
formatting/doc patch	reviewer + c gate
code/schema change	tester + auditor + c gate
release/publication	c gate + human gate
memory/core/privilege	c gate + human gate + witness
incident/legal-sensitive	human gate + evidence preservation + possible legal review

20.3 Judge-assistant limitation

A judge-assistant may compare, summarize, and recommend.

It does not decide.

---

21. Conformance levels

Level	Meaning
`CAPM-0`	No capability/permission separation
`CAPM-1`	Static permission list only
`CAPM-2`	Task-bound permissions with denied paths
`CAPM-3`	Capability profiles + task contracts + review separation
`CAPM-4`	Witnessed privilege transitions + drift detection
`CAPM-5`	High assurance: signed/canonical grants, revocation drills, cloud/local data split, conformance tests
`CAPM-X`	Revoked / non-conformant / prohibited autonomy

---

22. Mandatory conformance gates

Gate	Name	Blocking failure
G0	Capability/permission separation	capability treated as authority
G1	Deny-by-default	missing permission allowed
G2	Task-bound grants	broad standing permissions used
G3	Denied paths	no denied paths for material tasks
G4	Secrets default denied	secrets accessible by default
G5	Network bounded	unrestricted network allowed
G6	No self-approval	executor approves own work
G7	Memory protection	direct memory write allowed
G8	Core protection	core mutation without witness/human gate
G9	Drift detection	privilege drift not detected or ignored
G10	Red-line prohibition	prohibited permission granted

23. Red-line failures

A system MUST be classified as `CAPM-X` if:

1. an agent self-grants permission;
2. an agent obtains unrestricted filesystem access without explicit emergency gate;
3. an agent obtains unrestricted network access;
4. an agent reads or exports secrets outside scope;
5. an agent writes directly to `c` memory;
6. an agent modifies identity, witness, or privilege core without witness and human gate;
7. an agent approves its own work;
8. an agent performs or prepares live external retaliation;
9. an agent uses malware-like behavior;
10. an agent persists beyond task expiry without authorization;
11. a cloud agent receives sealed/private/legal material without explicit authorization;
12. permission drift becomes default behavior.

24. Reference default profiles

24.1 Safe cloud reader profile

```
profile_name: safe_cloud_reader
trust_level: TL-1
max_auto_connect_level: AC-3
allowed_permissions:
  - PERM-DISCOVER
  - PERM-READ-PUBLIC
  - PERM-READ-INTERNAL
prohibited_permissions:
  - PERM-WRITE-WORKTREE
  - PERM-SECRET-SCOPED
  - PERM-MEMORY-DIRECT-WRITE
  - PERM-FULL-NETWORK
data_policy:
  secrets_allowed: false
  private_memory_allowed: false
  sealed_material_allowed: false
  legal_material_allowed: false
  cloud_upload_allowed: false
```

24.2 Codex-like sandbox executor profile

```
profile_name: codex_like_sandbox_executor
trust_level: TL-3
max_auto_connect_level: AC-5
allowed_permissions:
  - PERM-READ-INTERNAL
  - PERM-WRITE-SANDBOX
  - PERM-WRITE-WORKTREE
  - PERM-EXEC-TEST
  - PERM-MEMORY-PROPOSE
required_controls:
  - task_contract
  - sandbox
  - branch_or_worktree
  - reviewer_required
  - no_self_approval
  - witness_for_write
prohibited_permissions:
  - PERM-SECRET-EXPORT
  - PERM-MEMORY-DIRECT-WRITE
  - PERM-CORE-DIRECT-MODIFY
  - PERM-LIVE-COUNTERATTACK
```

24.3 Local sentinel profile

```
profile_name: local_sentinel
trust_level: TL-2
max_auto_connect_level: AC-3
allowed_permissions:
  - PERM-READ-INTERNAL
  - PERM-INCIDENT-LOCAL
required_controls:
  - local_only
  - no_external_counteraction
  - witness_on_drift
  - human_gate_for_secret_rotation
prohibited_permissions:
  - PERM-LIVE-COUNTERATTACK
  - PERM-FULL-NETWORK
  - PERM-MALWARE-BEHAVIOR
```

24.4 High-risk core auditor profile

```
profile_name: high_risk_core_auditor
trust_level: TL-3
max_auto_connect_level: AC-3
allowed_permissions:
  - PERM-READ-INTERNAL
  - PERM-REVIEW
required_controls:
  - read_only
  - human_gate
  - witness
  - no_write
  - no_cloud_if_private_core
prohibited_permissions:
  - PERM-WRITE-WORKTREE
  - PERM-MEMORY-DIRECT-WRITE
  - PERM-CORE-DIRECT-MODIFY
```

25. Open issues

ID	Issue	Required action
OI-001	JSON Schema for capability profile	Create machine-readable `.schema.json`.
OI-002	JSON Schema for permission grant	Create machine-readable `.schema.json`.
OI-003	Provider-specific profiles	Define OpenAI/Codex, Google/Gemini, local agents, hybrid agents.
OI-004	Revocation drill profile	Define scheduled revocation and recovery tests.
OI-005	Permission event witness binding	Align with CLI Agent Witness Event Profile.
OI-006	Core authority surface registry	Define canonical list for Ester/Liya deployments.
OI-007	Cloud data policy split	Move detailed rules into Secrets and Cloud Data Policy companion.
OI-008	Drift scoring	Define exact D0-D5 scoring thresholds.
OI-009	UI indicators	Define how permissions are shown to human anchor and c.
OI-010	Repo placement	Decide final GitHub path and package index integration.

26. Closing rule

This model exists because CLI agents can do real work.

Real work requires real permission boundaries.

Final rule:

```
A capable agent is not an authorized agent.
An authorized agent is not a sovereign agent.
A sovereign c must be able to revoke every worker hand.
```